

ENGINEERING ASSESSMENT TASK

Build an AI Readiness Analyzer

A full-stack web application that scores any webpage on how well it is optimized for AI-powered search engines and answer bots.

STACK	EXTERNAL APIS	DURATION	COST TO BUILD
Your choice (Next.js preferred)	100% Free	3 – 5 Days	₹0 / \$0

01 — BACKGROUND

What is AI Readiness?

Modern AI tools like ChatGPT, Gemini, and Perplexity crawl the web and **cite specific webpages** when answering user questions. Whether a page gets cited depends on how well structured, readable, and discoverable it is — not just for humans, but for machines.

An **AI Readiness score** measures how "citation-ready" a webpage is. It checks things like: Does the page have proper headings? Is the content easy to read? Does it have structured data that machines can parse? Are Core Web Vitals good? Does it have a robots.txt and sitemap?

YOUR TASK

Build a web application where a user enters any URL, the system analyzes that page across multiple dimensions, computes an overall score (0–100) with a letter grade, and presents the results with actionable recommendations. Include an AI-powered chat interface so the user can ask questions about how to improve their page.

02 — TECH STACK

What to Use

You are free to choose your stack. Below are the recommendations — use what you know best, but the closer to these the better.

Frontend + Backend

Next.js 14 (App Router preferred) or any React + Node.js setup. The API routes should be part of the same project.

Database

PostgreSQL recommended. You can use Supabase (free tier) or any hosted Postgres. SQLite is acceptable for local-only.

Styling

Tailwind CSS preferred. Any CSS framework is fine. Focus on clean, readable UI — not decoration.

Language

TypeScript strongly preferred. JavaScript is acceptable but TypeScript shows better engineering judgment.

NO AUTHENTICATION NEEDED

You do not need to implement user login or sessions. All analyses are public/unprotected for this task. Focus on the core feature, not auth.

03 — FREE APIS TO INTEGRATE

The Two APIs You Will Integrate

Both APIs require only a free Google account. No billing, no credit card.

API 1 — Google PageSpeed Insights

Returns **Core Web Vitals** (LCP, CLS, FID, TTFB) and a Lighthouse performance score for any URL. This gives you real performance data directly from Google's infrastructure.

- ✓ Get key: **console.cloud.google.com** → **Enable "PageSpeed Insights API"** → **Create API Key**
- ✓ Free quota: 25,000 requests / day
- ✓ No billing required

```
GET https://www.googleapis.com/pagespeedonline/v5/runPagespeed
?url=https://example.com
&key=YOUR_API_KEY
&strategy=mobile

# Key response fields:
lighthouseResult.categories.performance.score → multiply by 100 → your CW
V score
lighthouseResult.audits['largest-contentful-paint'].displayValue
lighthouseResult.audits['cumulative-layout-shift'].displayValue
lighthouseResult.audits['total-blocking-time'].displayValue
```

API 2 — Google Gemini Flash 2.0

A fast, capable LLM from Google. You will use it for two things: (1) AI-enhanced content scoring — ask Gemini to evaluate 8 semantic dimensions of the page content, (2) optimization chat — user asks questions, Gemini answers using the analysis results as context.

- ✓ Get key: **aistudio.google.com/apikey** → Sign in → Create API Key
- ✓ Free tier: 15 requests/min, 1,500 requests/day
- ✓ Model to use: `gemini-2.0-flash`
- ✓ 1 million token context window

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flan-sh:generateContent?key=YOUR_KEY
```

Body:

```
{
  "contents": [{ "parts": [{ "text": "Your prompt here" }] }],
  "generationConfig": { "maxOutputTokens": 512, "temperature": 0.1 }
}
```

Response:

```
data.candidates[0].content.parts[0].text → the AI response text
```

ENVIRONMENT VARIABLES

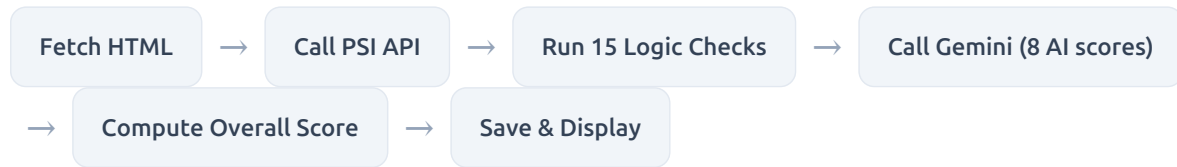
```
GOOGLE_PSI_API_KEY=...
```

```
GEMINI_API_KEY=...
```

04 — THE SCORING ENGINE

16 Checks to Implement

When a user submits a URL, your backend should: fetch the page HTML, call PSI, run all checks, then call Gemini for the AI-enhanced pass. Each check returns a score (0–100) and a status (`pass` / `warning` / `fail`).



#	CHECK NAME	TYPE	WHAT TO DETECT AND SCORE
1	Heading Hierarchy	LOGIC	Exactly one H1 = good. Multiple H1s or no H1 = penalize. Skipped heading levels (H1→H3) = penalize.
2	Content Readability	LOGIC	Compute Flesch Reading Ease score from extracted text. Higher = easier to read. Scale to 0–100.
3	Metadata Quality	LOGIC	Check for: page title, meta description (70–160 chars), OG tags (og:title, og:description), author meta, publish date.
4	Accessibility	LOGIC	Alt text on images (ratio of images with alt vs total). ARIA labels. lang attribute on <html>. Role attributes.
5	Semantic HTML	LOGIC	Count semantic HTML5 elements used: <article>, <nav>, <main>, <section>, <header>, <footer>, <aside>. More = higher score.
6	Structured Data	LOGIC	Find all <code><script type="application/ld+json"></code> blocks. Try JSON.parse each. Count valid vs invalid. All valid = 100.
7	FAQ & Q&A Structure	LOGIC	FAQPage JSON-LD schema = 100. HowTo schema = 50. <details> accordion with 3+ questions = 60. None = 0.
8	E-E-A-T Signals	LOGIC	Check for 6 trust signals: author meta tag, author markup in HTML, link to /about page, link to /contact page, author bio markup, datePublished. Score = (found / 6) × 100.
9	Content Freshness	LOGIC	Extract article:published_time and article:modified_time from meta tags or JSON-LD. Recently modified (within 1 year) = high score. Old or missing = low.
10	Table of Contents	LOGIC	Detect element with id/class "toc" or "table-of-contents" with 3+ anchor links, OR a <nav> with 3+ # links. Presence = pass.
11	Internal Linking	LOGIC	Count links pointing to the same domain. 0 = fail (score 0), 1–2 = warning (40–60), 3–4 = pass (80), 5+ = pass (100).

#	CHECK NAME	TYPE	WHAT TO DETECT AND SCORE
12	Outbound Link Health	LOGIC	Sample up to 5 external links. Send HEAD request to each (3s timeout). Count broken (4xx/5xx). 0 broken = 100, 1 = 60, 2+ = 30.
13	Robots.txt	LOGIC	Fetch /robots.txt. Present with User-agent and Sitemap directive = 100. Present without Sitemap = 80. Missing = 30.
14	XML Sitemap	LOGIC	Check /sitemap.xml and /sitemap_index.xml. Valid XML sitemap with 10+ URLs = 100. Found but small = 70. Missing = 20.
15	LLMs.txt File	LOGIC	Fetch /llms.txt. Present = 100 (pass). Missing = 0 (warning — this is optional but signals AI-awareness).
16	Core Web Vitals	PSI API	Use Google PSI API. Take the Lighthouse performance score (0.0–1.0), multiply by 100. This is your CWV score.

AI-Enhanced Pass (Gemini — 8 Additional Scores)

After the 16 checks, strip the HTML tags from the page content, truncate to ~3000 characters, and send to Gemini with this prompt:

You are an AI content evaluator. Analyze this web page content and score each metric 0-100.

URL: {url}

Content: {stripped_text_truncated_to_3000_chars}

Return ONLY a valid JSON object with exactly these 8 keys, integer values 0-100:

```
{
  "content_quality": 0-100,
  "info_architecture": 0-100,
  "semantic_structure": 0-100,
  "ai_discovery_value": 0-100,
  "knowledge_extraction": 0-100,
  "contextual_clarity": 0-100,
  "content_uniqueness": 0-100,
  "machine_interpretability": 0-100
}
```

Scoring guide:

- content_quality: Is content informative, accurate, and comprehensive?
- info_architecture: Are topics clearly organised with logical flow?
- semantic_structure: Does structure clearly reflect content hierarchy?
- ai_discovery_value: How likely would an AI cite this page in an answer?
- knowledge_extraction: How easily can facts/data be extracted from this page?
- contextual_clarity: Are key terms defined and concepts well explained?
- content_uniqueness: Does it offer original insights or perspectives?
- machine_interpretability: How easily can machines parse and understand the structure?

Parse the JSON from Gemini's response, then store these 8 as additional check rows with display labels like "Content Quality", "AI Discovery Value", etc.

Overall Score Formula

Weighted average across all checks. Suggested weights for the 16 core checks:

```
weights = {
  'core-web-vitals': 0.15,
  'meta-tags':      0.12,
  'structured-data': 0.10,
  'heading-structure': 0.10,
  'readability':     0.10,
  'eeat-signals':    0.08,
  'semantic-html':   0.07,
  'faq-structure':   0.07,
  'content-freshness': 0.06,
  'accessibility':   0.05,
  'robots-txt':      0.04,
  'sitemap':         0.04,
  'internal-linking': 0.03,
  'broken-links':    0.03,
  'toc-navigation':  0.03,
  'llms-txt':        0.03,
  // sum = 1.0
}

# AI-enhanced scores are shown separately – do not include in the overall score
# (they are bonus insight, not part of the core score)
```

Grade Mapping

```
score >= 90 → A+
score >= 85 → A
score >= 80 → A-
score >= 75 → B+
score >= 70 → B
score >= 65 → B-
score >= 60 → C+
score >= 55 → C
score >= 50 → C-
score >= 40 → D
score < 40 → F
```

05 — DATA STORAGE

Database Schema

Use any relational database (PostgreSQL recommended). You need two tables.

Table 1: `analyses`

One row per URL analysis. Stores the overall result.

```
CREATE TABLE analyses (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  url         TEXT NOT NULL,  
  overall_score INTEGER NOT NULL DEFAULT 0,    -- 0 to 100  
  grade       VARCHAR(2) NOT NULL,             -- 'A+', 'A', 'B', ... 'F'  
  status      VARCHAR(20) NOT NULL DEFAULT 'pending',  
              -- 'pending' | 'analyzing' | 'completed' | 'failed'  
  page_title  TEXT,                           -- extracted from HTML
```